

Atividade 08 - Monitoramento Analógico e Controle de PWM

Nome:	Lucas Daris de Souza
Matrícula:	202021250037
Turma:	Sistemas Embarcados 2026.1
Data:	05/05/2026

1. Parâmetros de configuração dos periféricos

Parâmetro ADC	Valor
Unidade ADC	ADC_UNIT_1
Canal ADC	ADC_CHANNEL_0
Resolução	12 bits
Atenuação	ADC_ATTEN_DB_6
Parâmetro PWM	Valor
Timer	LEDC_TIMER_0
Canal PWM	LEDC_CHANNEL_0
Frequência PWM	1000 Hz
Resolução PWM	12 bits
GPIO do LED	GPIO 11
Parâmetro Botão	Valor
GPIO	GPIO 20
Pull-up interno	Habilitado

Tipo de interrupção	GPIO_INTR_NEGEDGE
Debounce	200 ms

OBS:

1. Usado um PWM de 12 Bits e um ADC também de 12 Bits pra facilitar.
2. No PWM uma frequência 1000Hz pois facilita a visualização da mudança da luz.
3. A equação pra converter o valor bruto do ADC em tensão só foi necessária pra imprimir no monitor serial, como fica explicito na primeira observação.

Equação de Conversão ADC → Tensão

$$V = (ADC / 4095) \times 3.3$$

Onde:

- ADC: valor lido pelo conversor analógico-digital
- 4095: valor máximo em 12 bits
- 3.3V: tensão de referência do ESP32

2. Código Fonte Desenvolvido

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"
#include "driver/ledc.h"
#include "driver/gpio.h"
#include "esp_timer.h"

#define LED      11
#define CHANNEL  ADC_CHANNEL_0
#define BOTAO    20
volatile bool congelado =
false;
volatile int64_t last_interrupt_time = 0;
void IRAM_ATTR botao_isr(void
*arg)
{
    int64_t current_time =
esp_timer_get_time();

    // debounce de 200 ms
    if ((current_time - last_interrupt_time) > 200000)
    {
        congelado = !congelado;
        last_interrupt_time =
current_time;
    }
} void
pwm_init(void)
{
    // Timer LEDs → 1 kHz
    ledc_timer_config_t led_timer = {
        .speed_mode      = LEDC_LOW_SPEED_MODE,
        .timer_num        = LEDC_TIMER_0,
        .duty_resolution  = LEDC_TIMER_12_BIT,
        .freq_hz          = 1000,
```

```

        .clk_cfg          = LEDC_AUTO_CLK
    };
    ledc_timer_config(&led_timer);
    // LEDs (channel)
    ledc_channel_config_t ch = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .channel     = LEDC_CHANNEL_0,
        .timer_sel   = LEDC_TIMER_0,
        .gpio_num    = LED,
        .duty         = 0,
        .hpoint       = 0
    };
    ledc_channel_config(&ch);
} void
button_init(void)
{
    gpio_config_t io_conf
= {

        .pin_bit_mask = (1ULL << BOTAO),
        .mode = GPIO_MODE_INPUT,

        // pull-up interno
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,

        // interrupção na descida
        .intr_type = GPIO_INTR_NEGEDGE
    };
    gpio_config(&io_conf);

    gpio_install_isr_service(0);
    gpio_isr_handler_add(BOTAO, botao_isr,
NULL);
}
void app_main()
{
    printf("Hello, Wokwi!\n");
    pwm_init();
    button_init();
    adc_one_shot_unit_handle_t
adc1_handle;
    adc_one_shot_unit_init_cfg_t init_config_adc1 =
{
        .unit_id = ADC_UNIT_1,
    };

```

```

    adc_oneshot_new_unit(&init_config_adc1, &adc1_handle);
    // Configuração dos canais      adc_oneshot_chan_cfg_t
config_adc_channel = {
    .bitwidth = ADC_BITWIDTH_12,
    .atten = ADC_ATTEN_DB_12
};    adc_oneshot_config_channel(adc1_handle, CHANNEL, &config_adc_channel);
    int value_adc_1 = 0;
    while (true)
    {
        if (!congelado)
        {
            adc_oneshot_read(adc1_handle, CHANNEL,
&value_adc_1);
            ledc_set_duty(LED_C_LOW_SPEED_MODE,    LEDC_CHANNEL_0,    value_adc_1);
            ledc_update_duty(LED_C_LOW_SPEED_MODE, LEDC_CHANNEL_0);    }
            printf("ADC: %d | Tensao: %.2f V | %s\n",
value_adc_1,
                (value_adc_1 / 4095.0) * 3.3,
congelado ? "CONGELADO" : "ATIVO");
            vTaskDelay(pdMS_TO_TICKS(100));
        }
    }
}

```